Machine Learning from Zero to Hero: Visual Companion

Graphviz diagrams and Manim animations that go with the written report. Part 1 holds eight diagrams. Part 2 holds six animated scenes; a PDF cannot play video, so each animation is shown here as a filmstrip of its frames, in order, left to right and top to bottom.

Contents

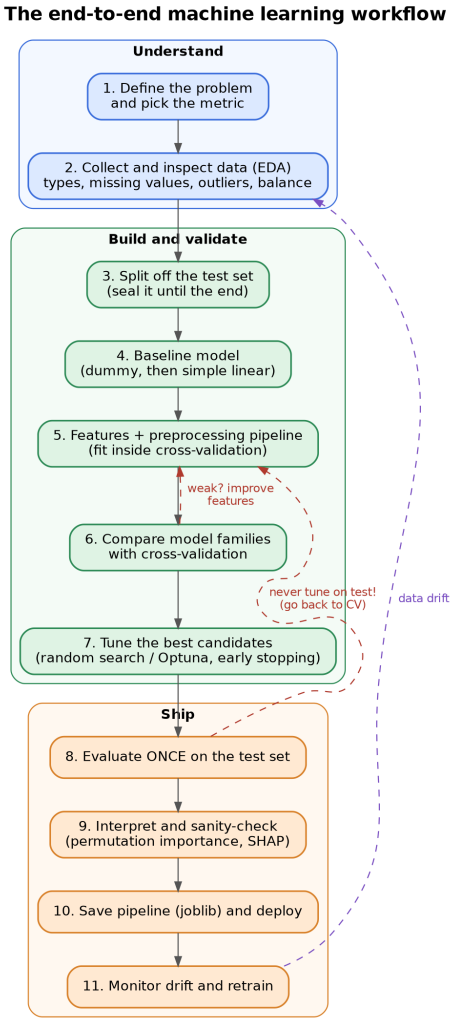
Part 1: The end-to-end workflow; A map of classical machine learning; Which algorithm should I try first?; Data splitting and 5-fold cross-validation; Two ways to combine trees; A small decision tree; Gradient descent: the learning loop; A scikit-learn Pipeline with ColumnTransformer.

Part 2: Gradient descent: converge vs. diverge; Linear regression by gradient descent; Logistic regression: the boundary learns; k-means: assign, move, repeat; PCA: project onto the direction of greatest variance; Underfitting vs. overfitting.

Part 1. Graphviz diagrams

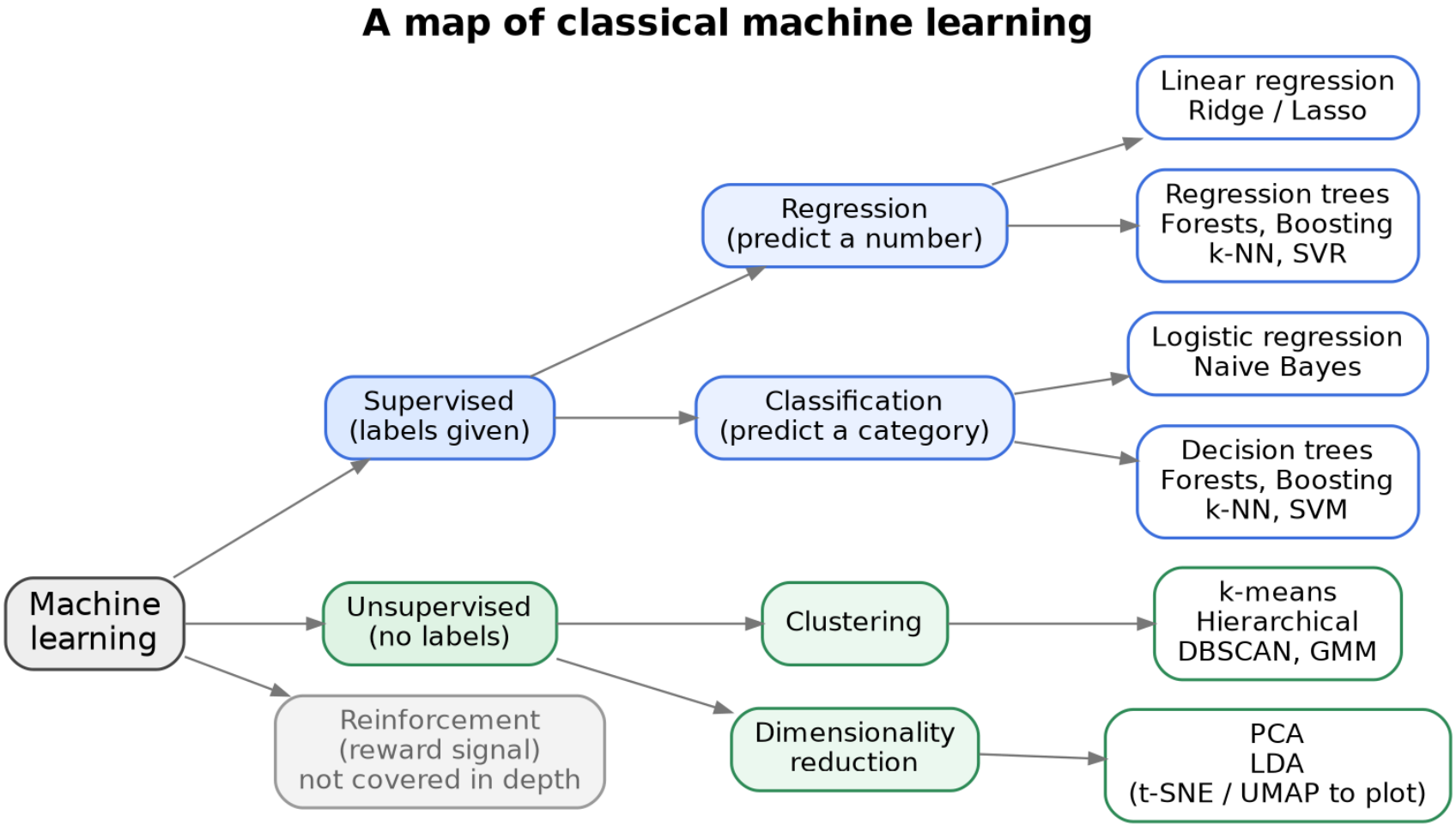
Sources are in graphviz/*.dot (re-render with: dot -Tpng -Gdpi=170 file.dot -o file.png). SVG versions are included for crisp scaling.

The end-to-end workflow



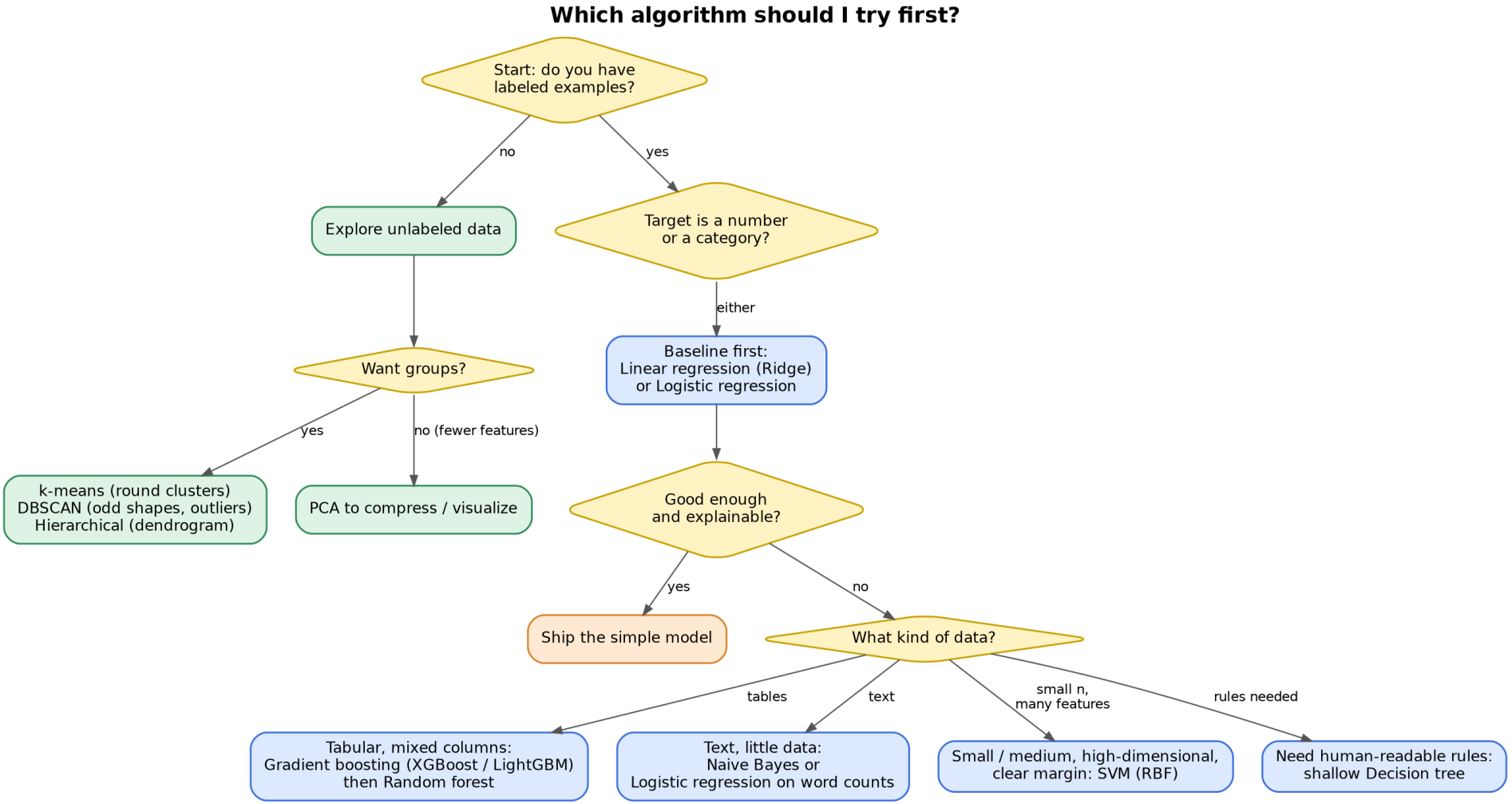
Section 32. Dashed arrows are the loops that matter: weak validation scores send you back to features, never to the test set.

A map of classical machine learning



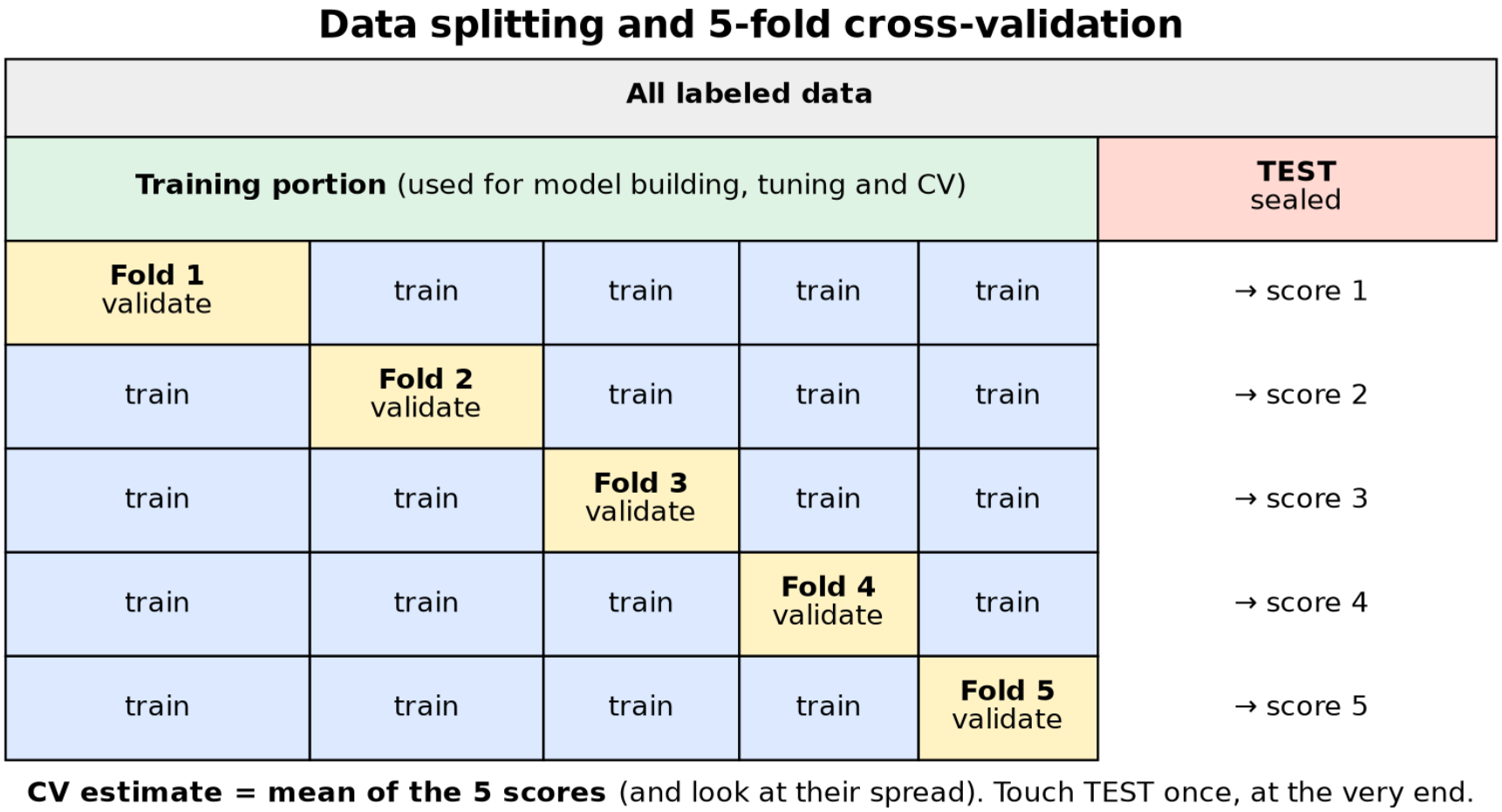
Sections 1 and 2, and the algorithms of Part B.

Which algorithm should I try first?



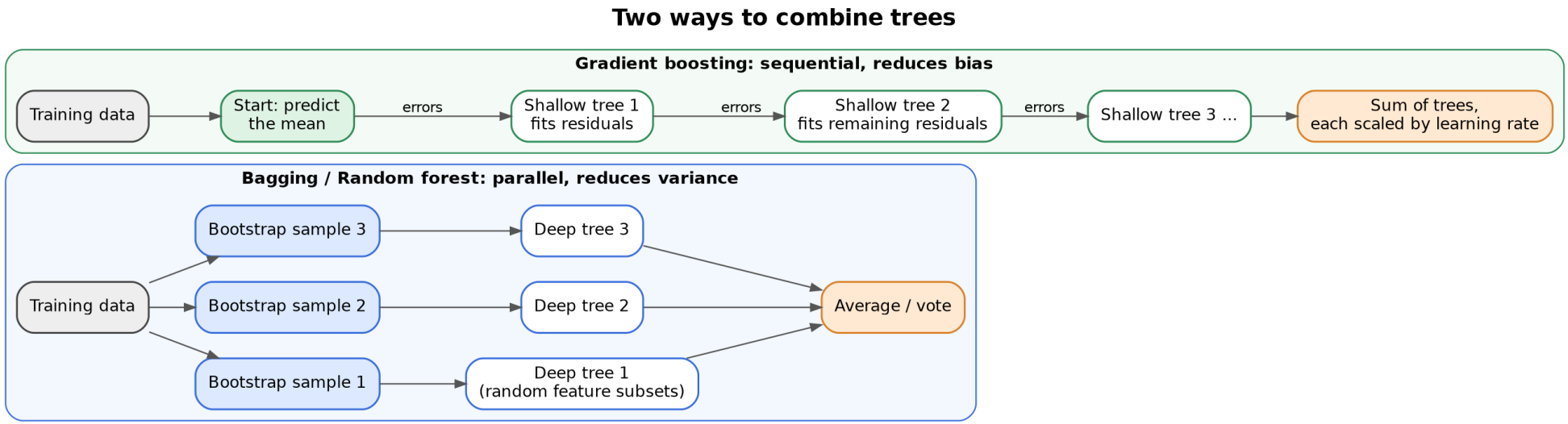
Section 20 summary map, as a decision flowchart. Always start with a simple baseline.

Data splitting and 5-fold cross-validation



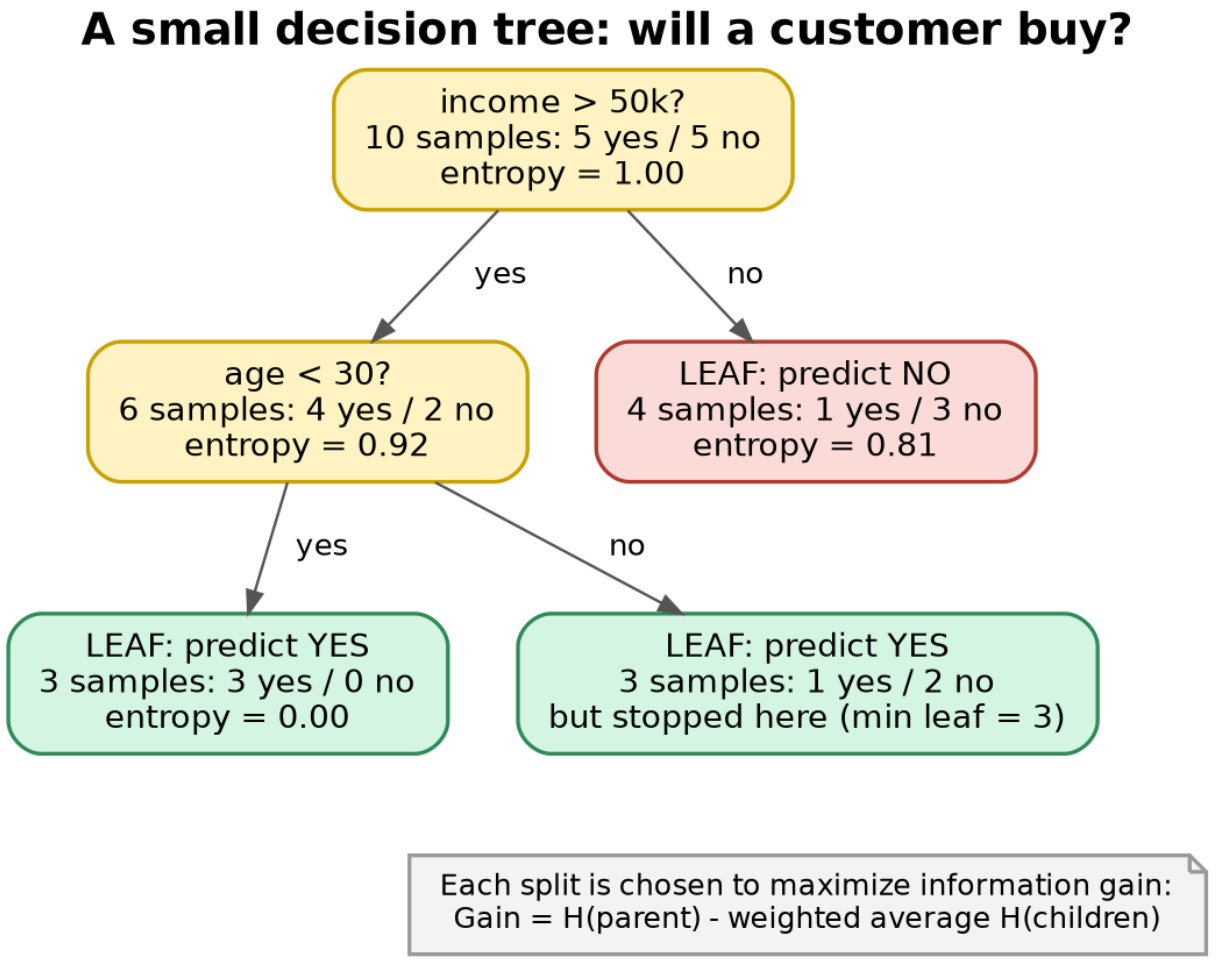
Sections 8 and 20. Every sample is used for validation exactly once; the test set stays sealed.

Two ways to combine trees



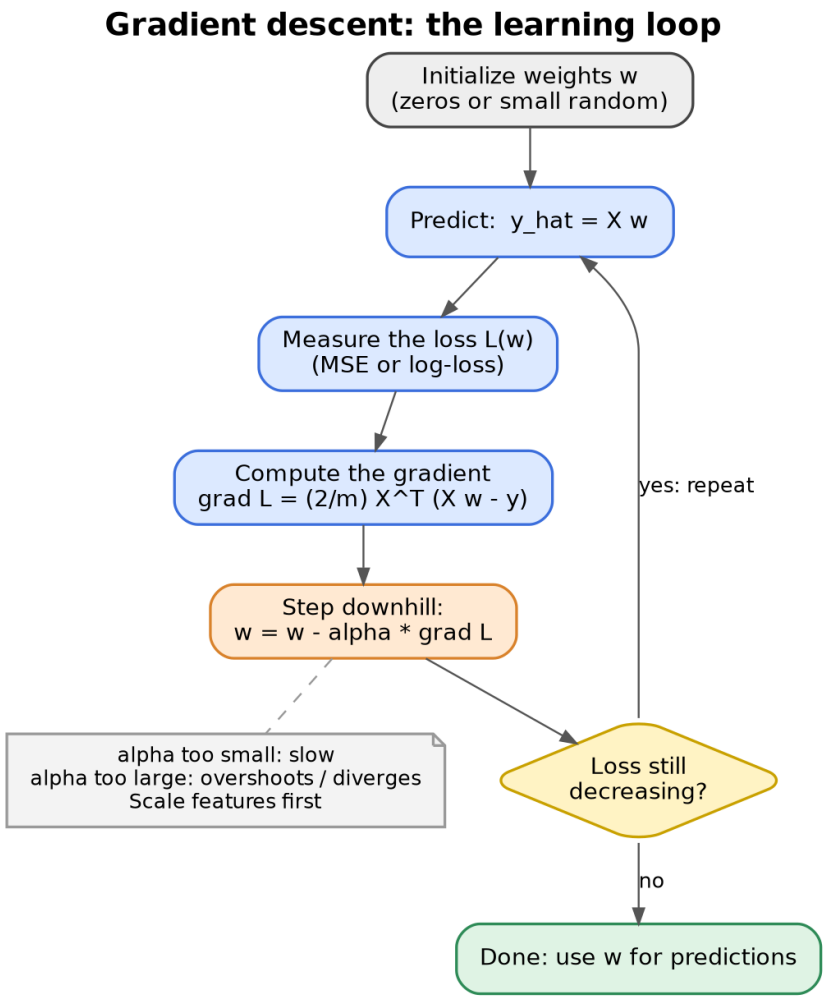
Sections 15 and 16. Bagging averages independent deep trees (cuts variance); boosting adds shallow trees one after another (cuts bias).

A small decision tree



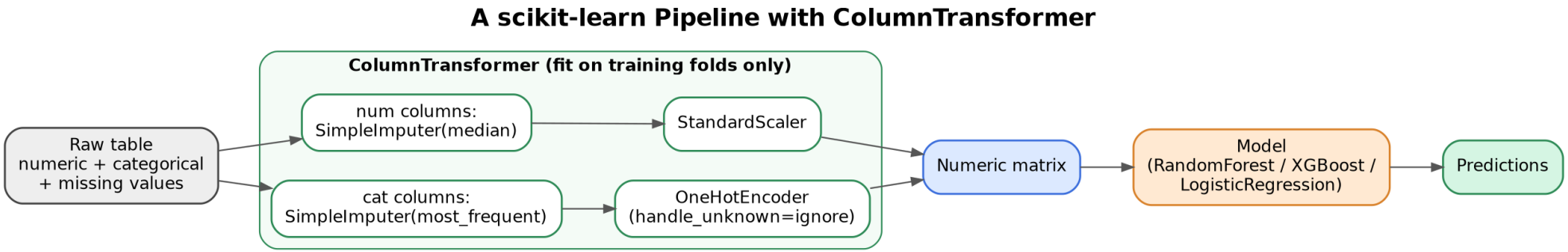
Section 14. Entropy values use H = -sum p log2 p; the best split maximizes information gain.

Gradient descent: the learning loop



Sections 5, 9 and 10.

A scikit-learn Pipeline with ColumnTransformer



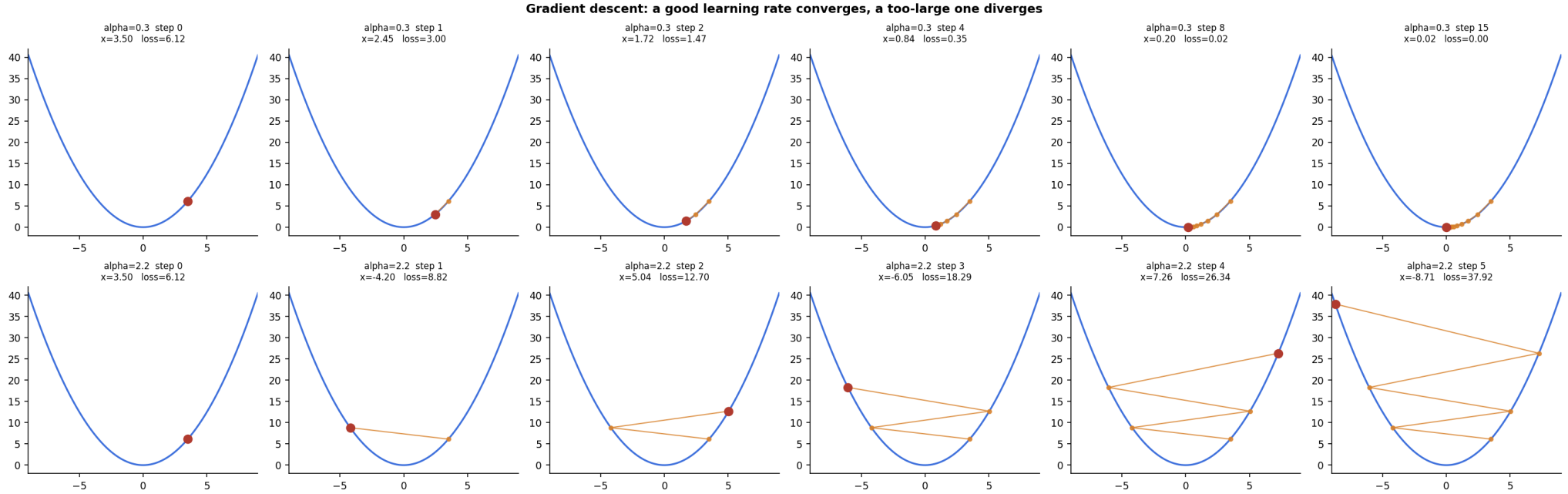
Sections 25 and 26. Everything inside is fitted on training folds only, which prevents leakage.

Part 2. Manim animations as filmstrips

Each page: a short description of what to watch, then the frames. Where a rendered Manim MP4 is present, the frames are real Manim output; otherwise a storyboard drawn with matplotlib from the same seeds, data and hyper-parameters is shown (labelled below each strip).

Gradient descent: converge vs. diverge (report: Sections 5 and 9; scene: GradientDescentScene)

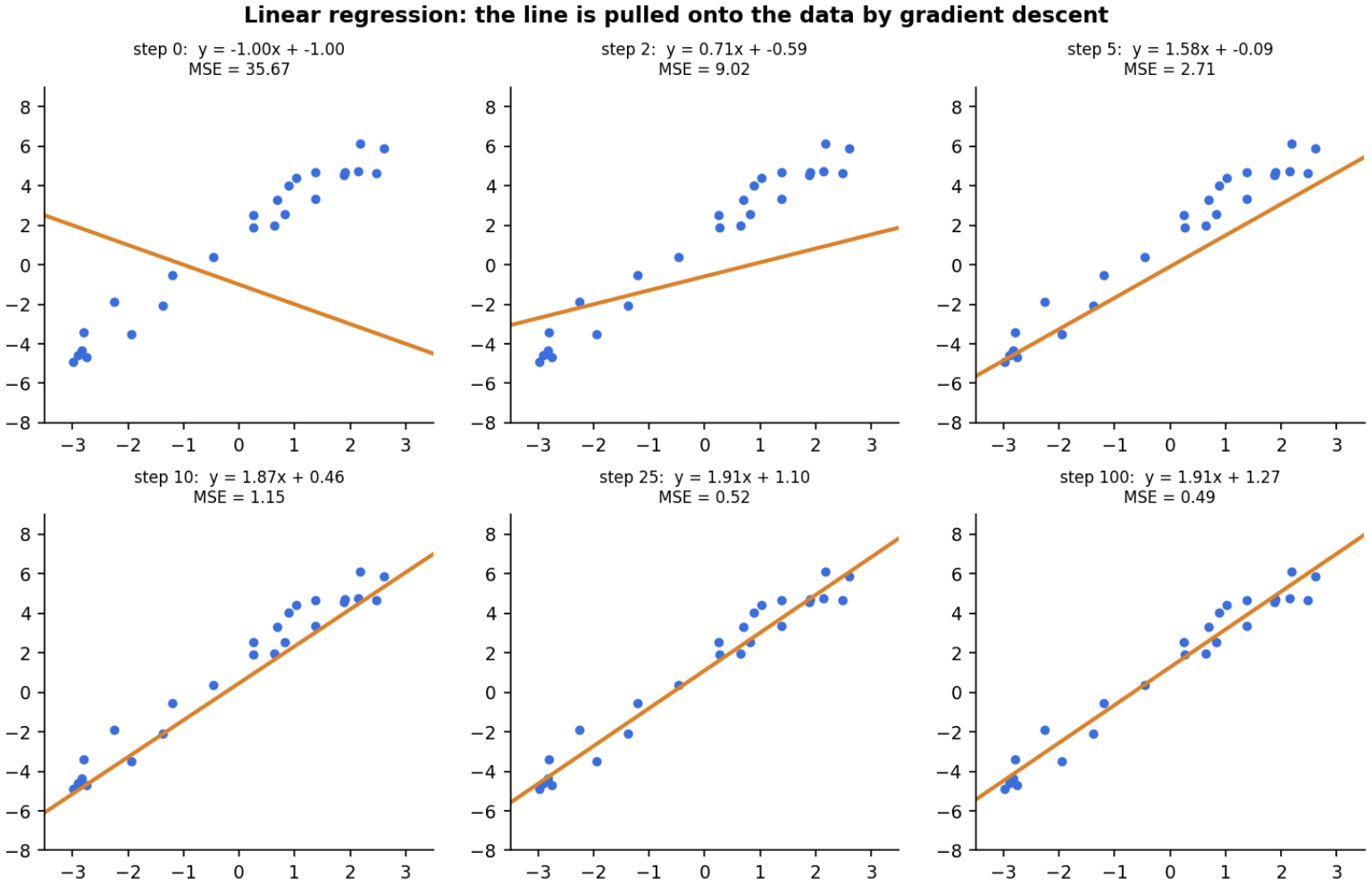
A dot slides down the loss curve f(x) = x^2/2. With alpha = 0.3 each step shrinks x by 30% and the dot settles in the valley. With alpha = 2.2 each step overshoots by more than it corrects, so x flips sign and grows: divergence.



Storyboard preview (matplotlib; identical algorithm, data and numbers to the Manim scene). Render the scene with Manim to get the true animation.

Linear regression by gradient descent (report: Section 9; scene: LinearRegressionScene)

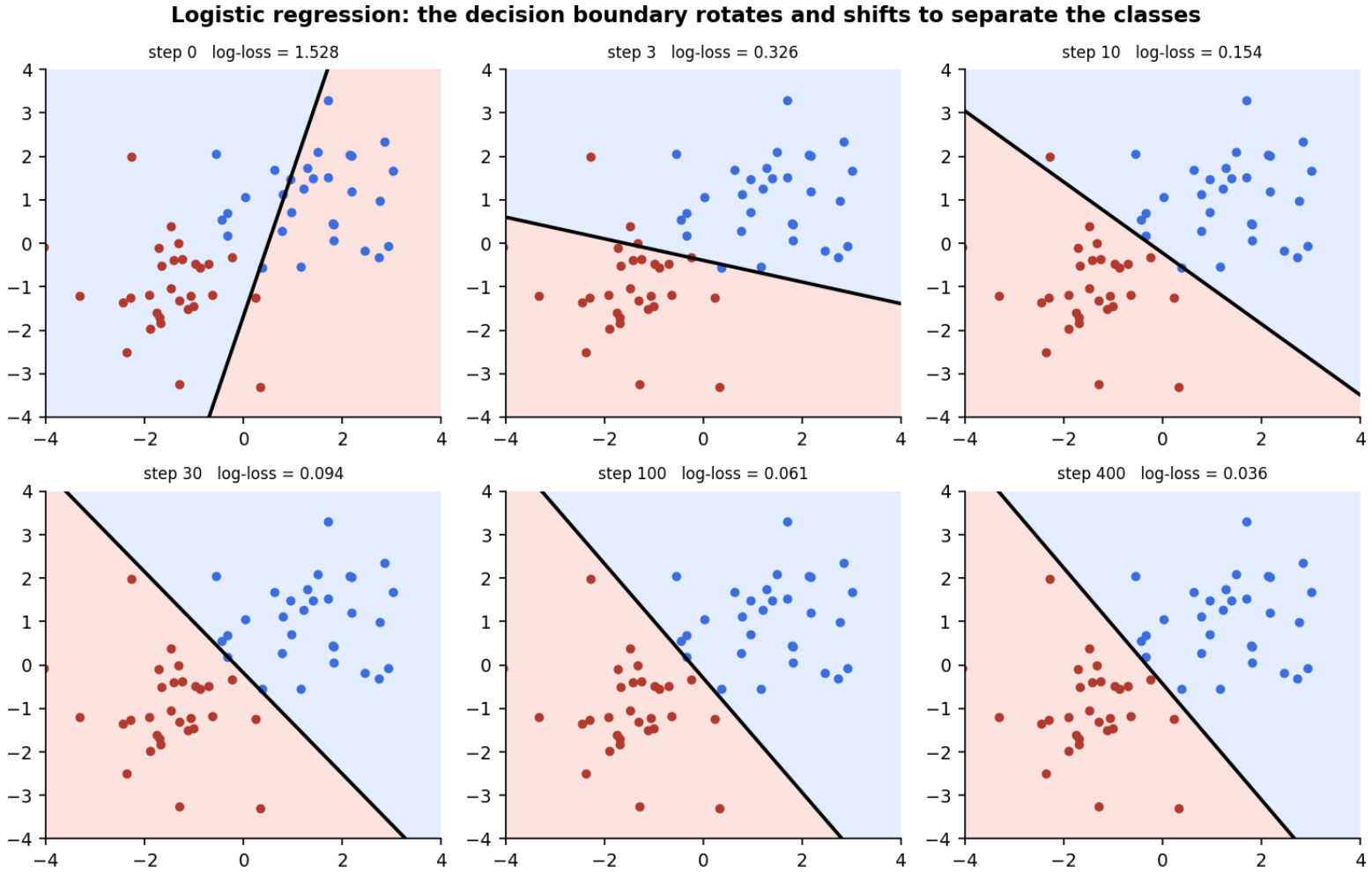
Twenty-five noisy points were generated from y = 1.8x + 1. The line starts at a bad guess (slope -1, intercept -1) and is pulled onto the cloud; the MSE printed underneath falls at every step.



Storyboard preview (matplotlib; identical algorithm, data and numbers to the Manim scene). Render the scene with Manim to get the true animation.

Logistic regression: the boundary learns (report: Section 10; scene: LogisticBoundaryScene)

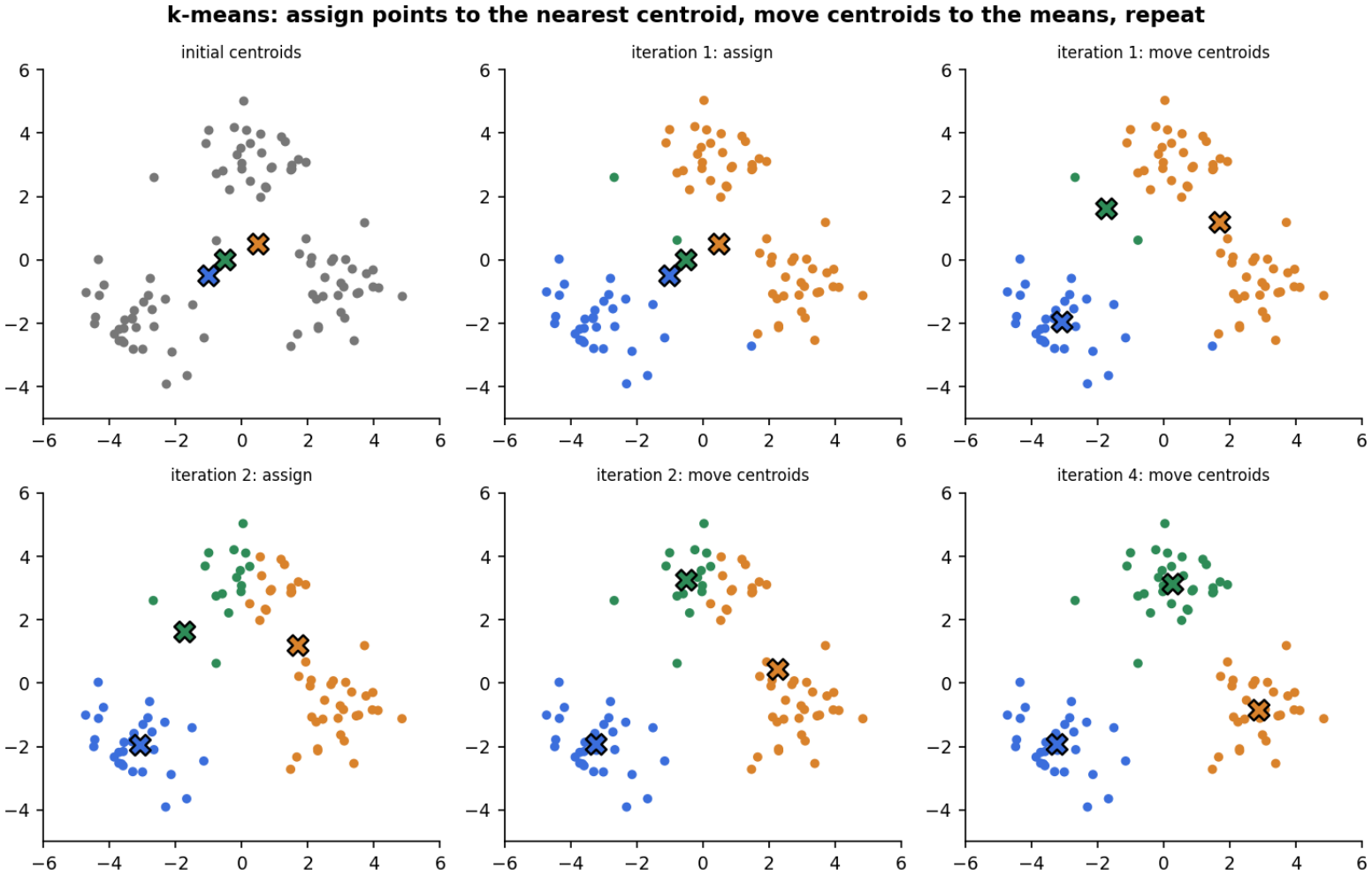
Two classes of 30 points each. The white line is where w.x + b = 0 (probability exactly 0.5). It starts badly oriented, rotates and shifts as log-loss falls, and ends between the classes.



Storyboard preview (matplotlib; identical algorithm, data and numbers to the Manim scene). Render the scene with Manim to get the true animation.

k-means: assign, move, repeat (report: Section 18; scene: KMeansScene)

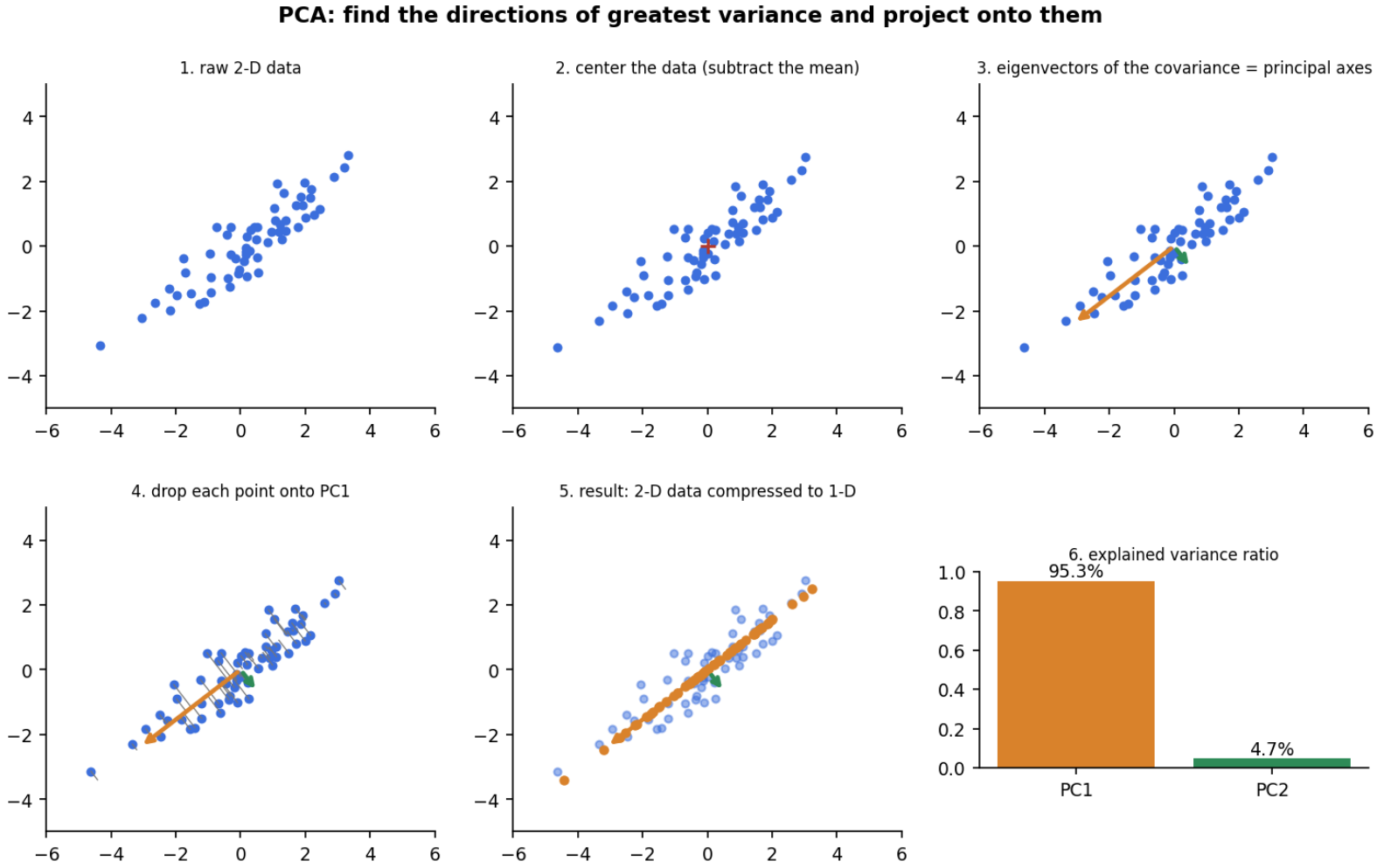
Three centroids start clumped in the middle. Each iteration has two beats: points take the colour of the nearest centroid, then each centroid jumps to the mean of its points. Stop when nothing changes.



Storyboard preview (matplotlib; identical algorithm, data and numbers to the Manim scene). Render the scene with Manim to get the true animation.

PCA: project onto the direction of greatest variance (report: Section 19; scene: PCAScene)

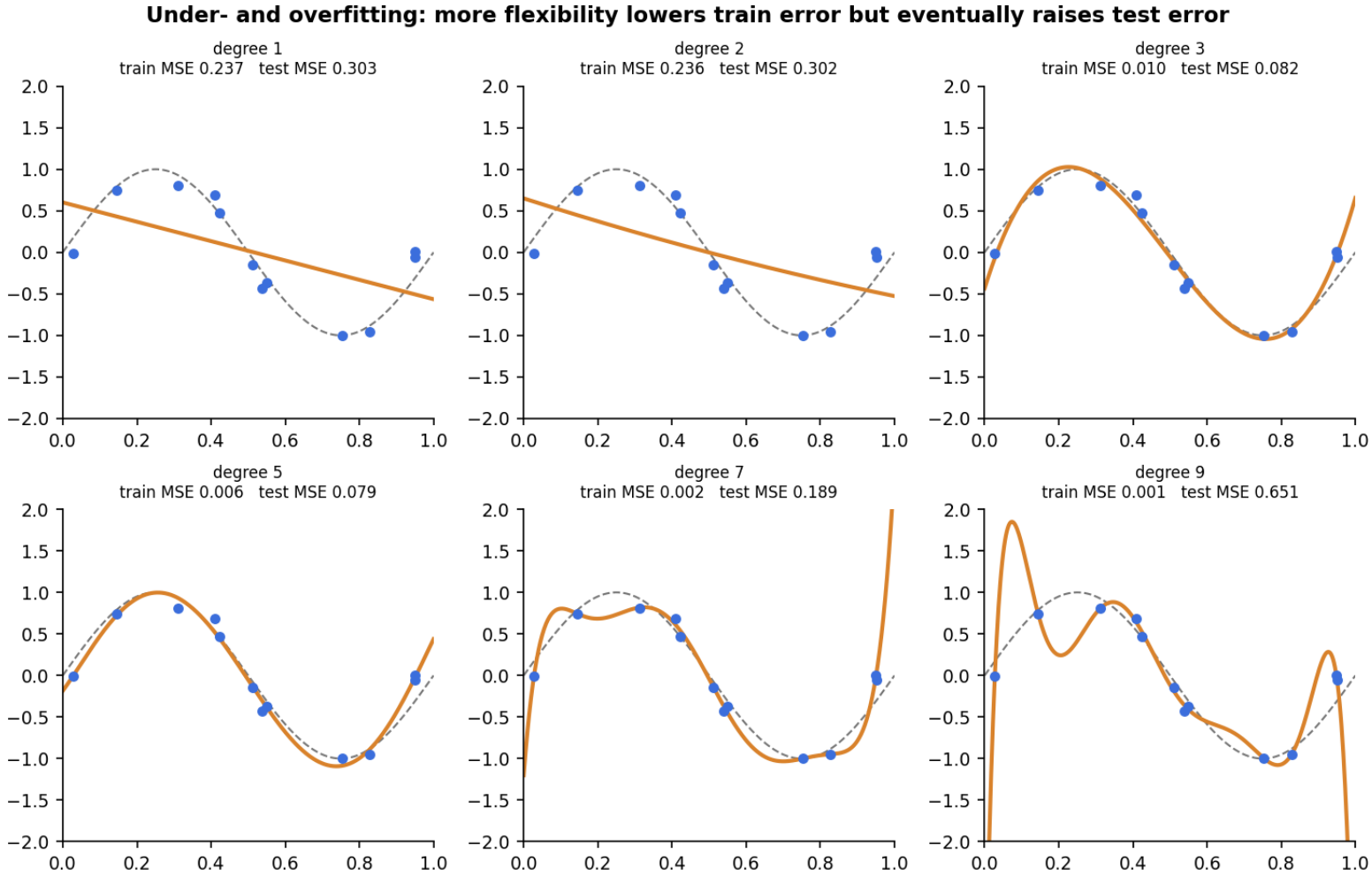
Correlated 2-D data is centered, the two eigenvectors of the covariance matrix are drawn as axes (longer arrow = more variance), and every point is dropped onto PC1, compressing 2-D to 1-D.



Storyboard preview (matplotlib; identical algorithm, data and numbers to the Manim scene). Render the scene with Manim to get the true animation.

Underfitting vs. overfitting (report: Sections 8 and 11; scene: OverfittingScene)

12 noisy samples of a sine wave. Raising the polynomial degree always lowers training error, but test error is lowest near degree 3 to 5 and climbs again as the curve starts chasing noise.



Storyboard preview (matplotlib; identical algorithm, data and numbers to the Manim scene). Render the scene with Manim to get the true animation.

How to make the animations play, and how to rebuild this PDF with the real Manim frames

```
1. pip install manim   (Python 3.9+, ffmpeg installed)
2. cd manim && ./render_all.sh        # writes MP4s to manim/media/videos/ml_scenes/720p30/
3. python build/build_companion_pdf.py   # re-extracts frames from those MP4s and rebuilds this PDF
4. (optional) playable animation inside the PDF: python build/make_animated_tex.py  then  pdflatex animated_scenes.tex
```

About playable PDFs: PDF has no universal video support. The LaTeX animate package embeds the frames as a JavaScript slideshow that plays in Adobe Acrobat Reader and a few other viewers; browsers, Preview and most phone viewers show only the first frame, which is why the filmstrips above are the portable form. The MP4 files always play everywhere.